

Vega-Lite

Drawing Charts with JSON

Information Visualization · Lecture 5

Create and modify scatterplots, bar charts, histograms, and line charts.

Vega-Lite describes charts with JSON

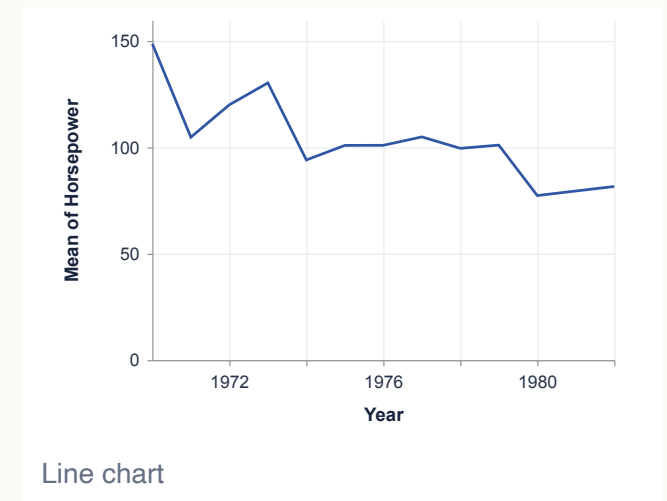
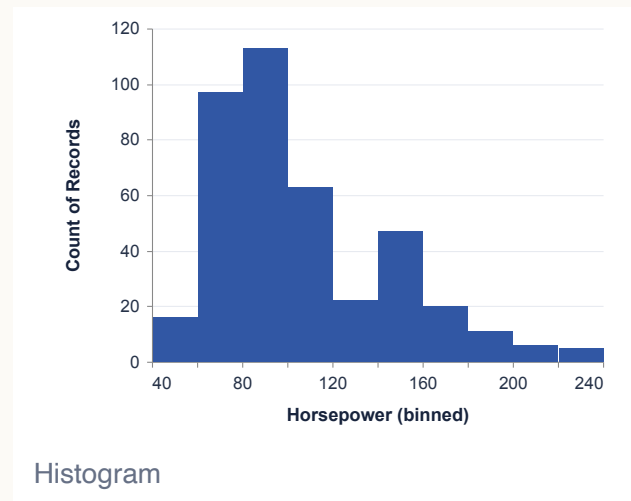
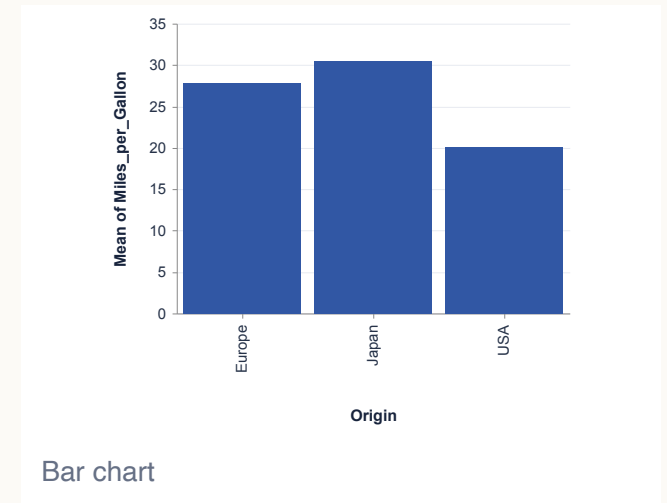
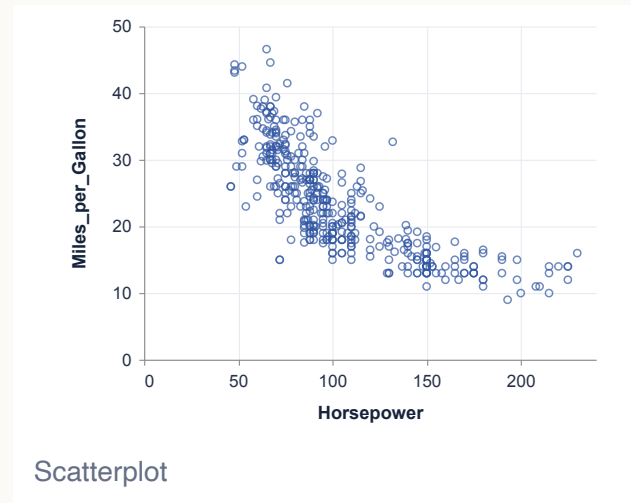
Vega-Lite is a declarative visualization language. A JSON specification describes the data, graphical marks, and visual channels that can encode data fields.

For example, declare that `Horsepower` controls horizontal position. Vega-Lite computes the positions and draws an axis.

The same language can describe all four charts shown here.

Reference: [Vega-Lite · Overview](#)

Charts: [Vega Datasets · Cars](#)



Today's agenda

- | | | |
|----|--------------------------------------|---|
| 01 | A first scatterplot | Read the data, mark, and encoding structure. |
| 02 | Marks and channels | Explore common choices, then modify the scatterplot. |
| 03 | Prepare data for other charts | Use aggregation and binning to derive values to encode. |
| 04 | Write and run code | Adapt examples and check the resulting chart. |

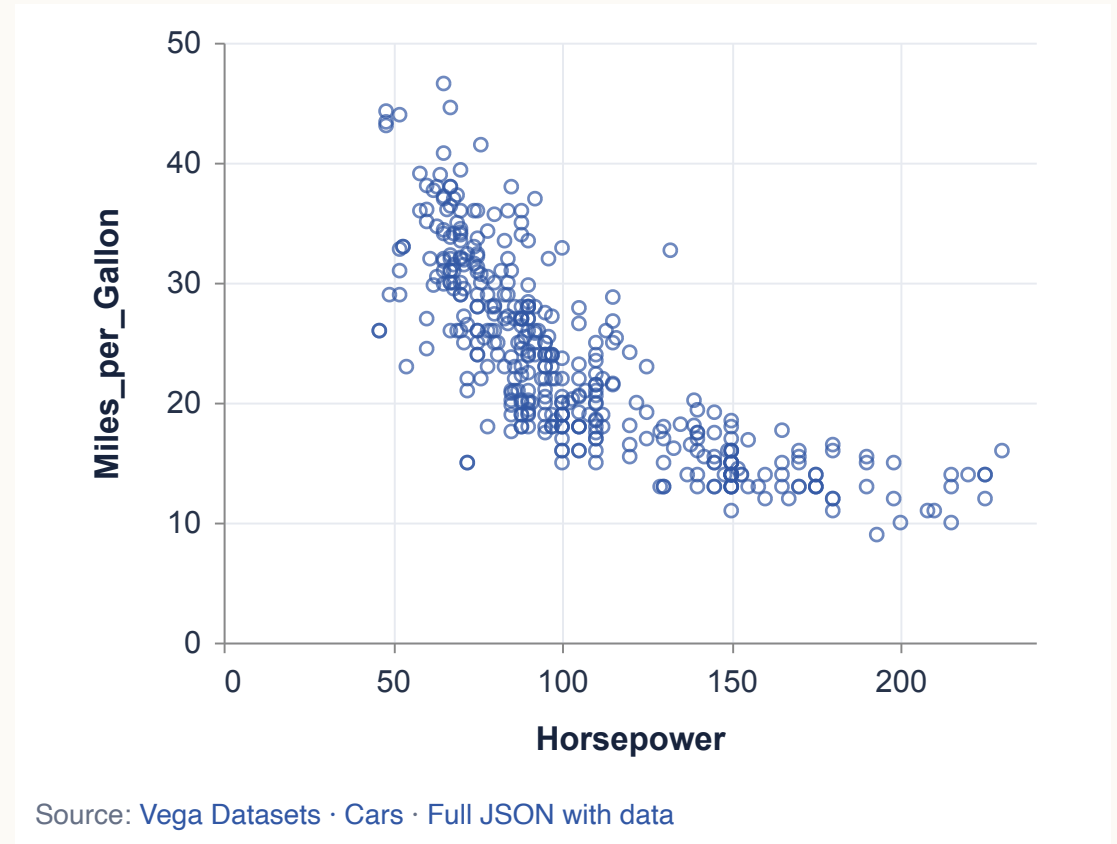
A first scatterplot

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "point",
  "encoding": {
    "x": {
      "field": "Horsepower",
      "type": "quantitative"
    },
    "y": {
      "field": "Miles_per_Gallon",
      "type": "quantitative"
    }
  }
}
```

To run: open [Vega Editor](#) and paste this specification.

Vega Editor resolves data/cars.json. The linked JSON also includes the data.



Each point shows Horsepower and MPG for a car with both values present.

Reference: [Vega-Lite · Spec](#)

Three properties define this scatterplot

INSIDE THE TOP-LEVEL OBJECT

```
"data": {"url": "data/cars.json"},
"mark": "point",

"encoding": {
  "x": {
    "field": "Horsepower",
    "type": "quantitative"
  },
  "y": {
    "field": "Miles_per_Gallon",
    "type": "quantitative"
  }
}
```

data

The source records. Each car has named fields such as Horsepower.

mark

The graphical shape. Here, `"point"` draws the scatterplot's symbols.

encoding

Data mappings or fixed channel values. Here, Horsepower maps to x and MPG to y.

JSON: braces `{ }` enclose objects, brackets `[ ]` enclose arrays. Separate properties with commas and put keys and strings in double quotes.

Reference: [Vega-Lite](#) · [Spec](#)

Common marks provide graphical building blocks

A **mark** is a graphical element that represents data. The `mark` property selects its type.

mark value	Appearance	What it draws
"point"		Symbols with a configurable shape.
"circle"		Filled circular symbols.
"bar"		Bars whose length can encode a value.
"line"		Line segments connecting ordered positions.
"area"		Filled regions between two boundaries.
"rect"		Rectangular cells or regions.
"text"	Aa Bb Cc	Text labels at encoded positions.

Schematic glyphs illustrate shapes. Common choices shown here; see [Vega-Lite · Mark](#) for the full list.

Common channels belong inside encoding

Channels control visual properties of marks. Define each channel as a property inside `encoding`.

Channel	What it controls	Example inside encoding
<code>x / y</code>	Position	<pre>"x": {"field": "Horsepower", "type": "quantitative"}</pre>
<code>color</code>	Color	<pre>"color": {"field": "Origin", "type": "nominal"}</pre>
<code>size</code>	Symbol area for points	<pre>"size": {"value": 80}</pre>
<code>shape</code>	Symbol form	<pre>"shape": {"field": "Origin", "type": "nominal"}</pre>
<code>opacity</code>	Transparency	<pre>"opacity": {"value": 0.5}</pre>

`field` + `type` maps data; `value` sets a constant.

Available channels depend on the mark. Use `shape` with `point`.

Channel names link to full Cars examples. Reference: [Vega-Lite · Encoding](#).

Data can be inline or loaded from a URL

Inline records: values

TWO REAL CARS RECORDS

```
"data": {
  "values": [
    {"Horsepower": 130,
     "Miles_per_Gallon": 18},
    {"Horsepower": 165,
     "Miles_per_Gallon": 15}
  ]
}
```

Each object is one record. Its property names become fields.

External file: url

ALL 406 CARS RECORDS

```
"data": {
  "url": "data/cars.json"
}
```

A URL can identify a JSON or CSV file. In Vega Editor, use `data/cars.json` to load Cars.

Field names must match exactly:

`Miles_per_Gallon` is the data field; `MPG` is only our spoken shorthand.

Reference: [Vega-Lite · Data](#)

Records: [Vega Datasets · Cars](#)

A circle mark can use a fixed opacity

Changing `point` to `circle` gives the scatterplot filled circular marks. Set their opacity inside `encoding`.

CHANGE MARK; ADD OPACITY TO ENCODING

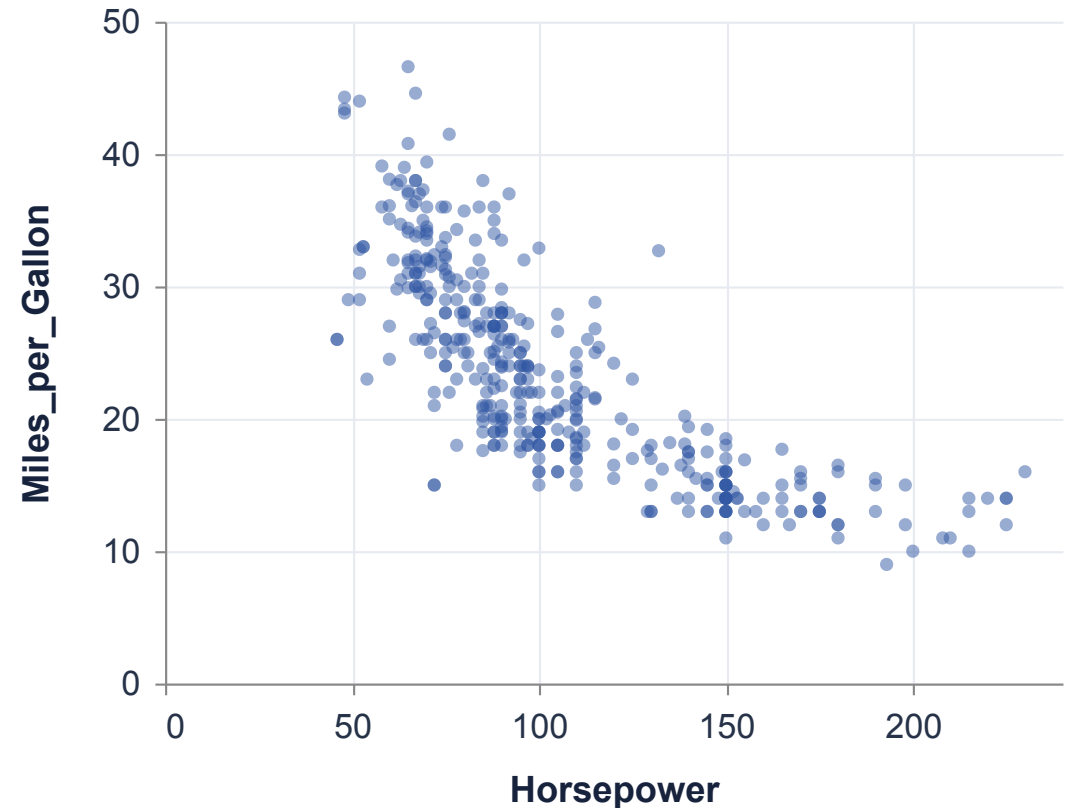
```
"mark": "circle",  
"encoding": {  
  "opacity": {"value": 0.5}  
}
```

`circle` selects the mark type.

`opacity` is a channel; `value` sets the same opacity for every circle.

Keep the existing x and y definitions inside encoding.

Reference: [Vega-Lite · Mark](#)



Source: [Vega Datasets · Cars · Full JSON with data](#)

Encoding describes each visual channel

Channels control visual properties of marks, such as position, color, size, and opacity.

`encoding` describes how each channel gets its values. It can map a data field or assign a fixed visual value.

INSIDE ENCODING

```
"x": {  
  "field": "Horsepower",  
  "type": "quantitative"  
}
```

Reference: [Vega-Lite · Encoding](#)

Channel: x

Use horizontal position.

Field: Horsepower

Read each car's Horsepower value.

Type: quantitative

Treat the values as numeric amounts.

Vega-Lite derives a scale and axis from the declaration. The `y` encoding has the same structure.

Changing a field changes the x-axis

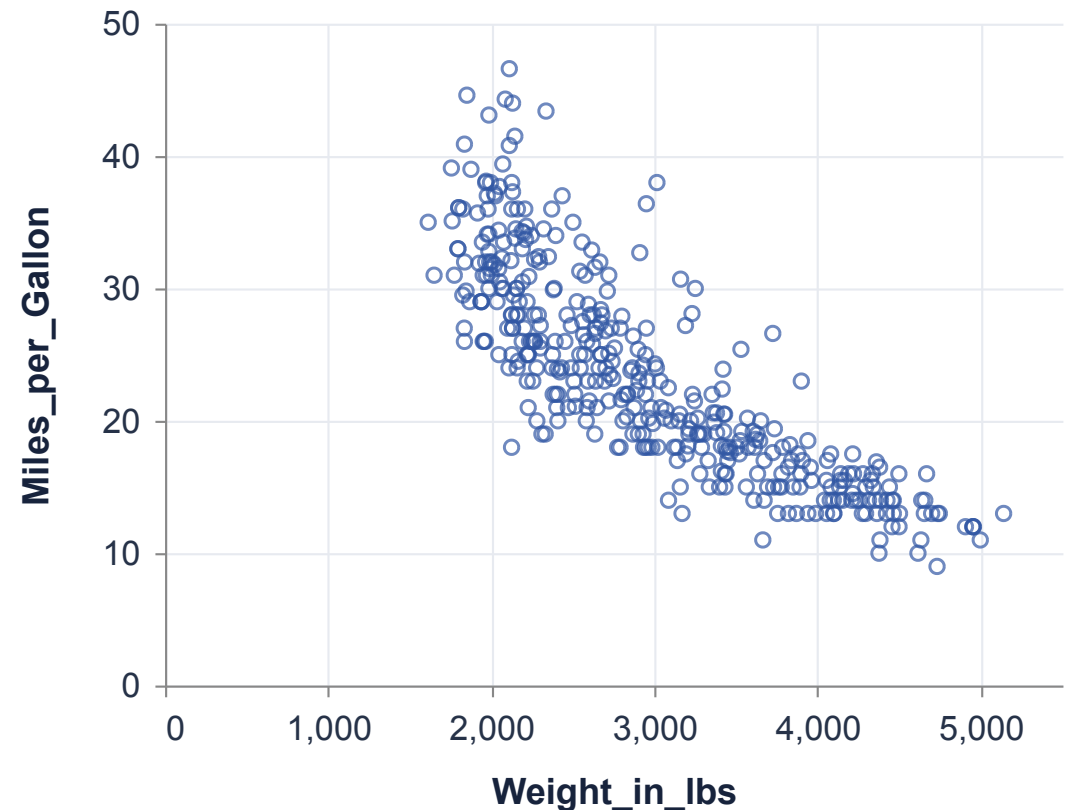
REPLACE X INSIDE ENCODING

```
"x": {  
  "field": "Weight_in_lbs",  
  "type": "quantitative"  
}
```

The x-axis now shows vehicle weight in pounds. The y-axis still shows MPG.

The data source and point mark remain the same. The axis title and scale update from the new field.

Reference: [Vega-Lite · Encoding](#)



Source: [Vega Datasets · Cars · Full JSON with data](#)

Field types tell Vega-Lite how to interpret values

The `type` property declares the meaning of values in an encoding. It helps Vega-Lite choose scales, guides, and formatting.

Vega-Lite type	Interpretation	Example
<code>nominal</code>	Categories with no inherent order	Origin: Europe, Japan, USA
<code>ordinal</code>	Categories with a meaningful order	Low, medium, high
<code>quantitative</code>	Numeric amounts or measurements	Horsepower: 95, 130, 165

`temporal` handles dates and times, with time-aware scales and formatting.

Cars has a `Year` field such as `"1970-01-01"`. We will use it for a line chart.

Reference: [Vega-Lite · Type](#)

Color encodes Origin and adds a legend

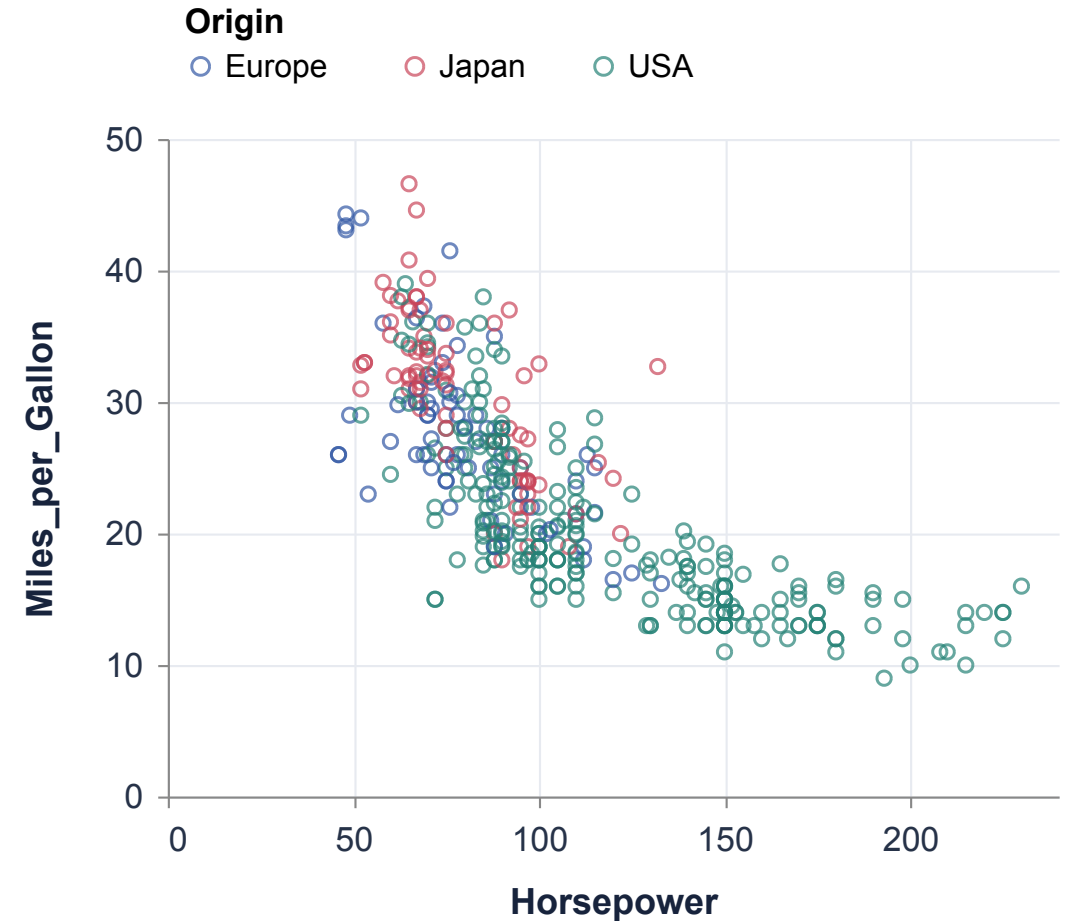
ADD A PROPERTY INSIDE ENCODING

```
"color": {  
  "field": "Origin",  
  "type": "nominal"  
}
```

Cars from the same Origin share a color. The legend identifies the three groups.

Use `nominal` because Origin names categories with no inherent ordering.

Reference: [Vega-Lite · Encoding](#)

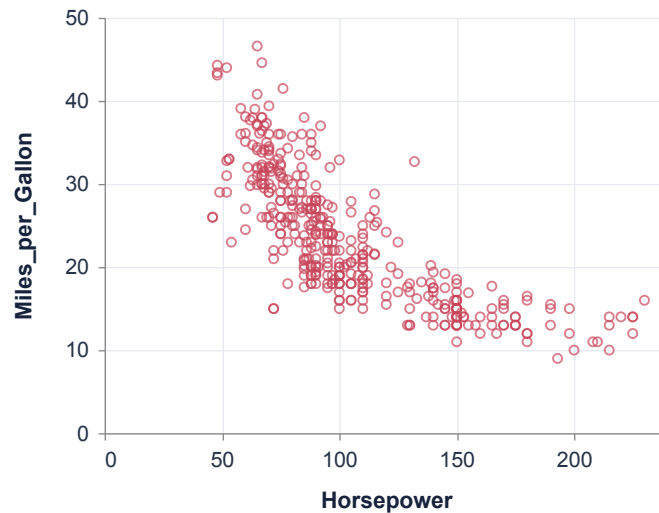


Source: [Vega Datasets · Cars · Full JSON with data](#)

A channel can use a constant or a data field

INSIDE ENCODING · A FIXED VISUAL VALUE

```
"color": {  
  "value": "#c9455a"  
}
```

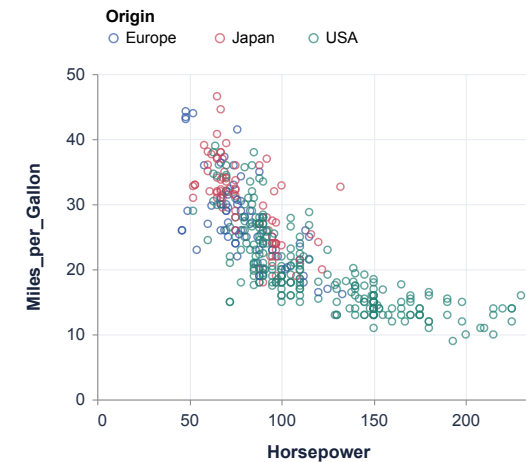


Source: [Vega Datasets · Cars · Full JSON with data](#)

`value` gives every point the same color. No color legend is needed.

INSIDE ENCODING · A DATA FIELD MAPPING

```
"color": {  
  "field": "Origin",  
  "type": "nominal"  
}
```



Source: [Vega Datasets · Cars · Full JSON with data](#)

`field` and `type` map Origin to color and produce a legend.

Practice: modify the scatterplot

Create a chart of **weight versus horsepower**, with colors for Origin.

1. Set x to `Weight_in_lbs`.
2. Set y to `Horsepower`.
3. Use `circle` marks.
4. Add the Origin color encoding.

Run it and check

Both position fields are quantitative. Origin is nominal.

Check the two axis titles, filled symbols, and three-category legend.

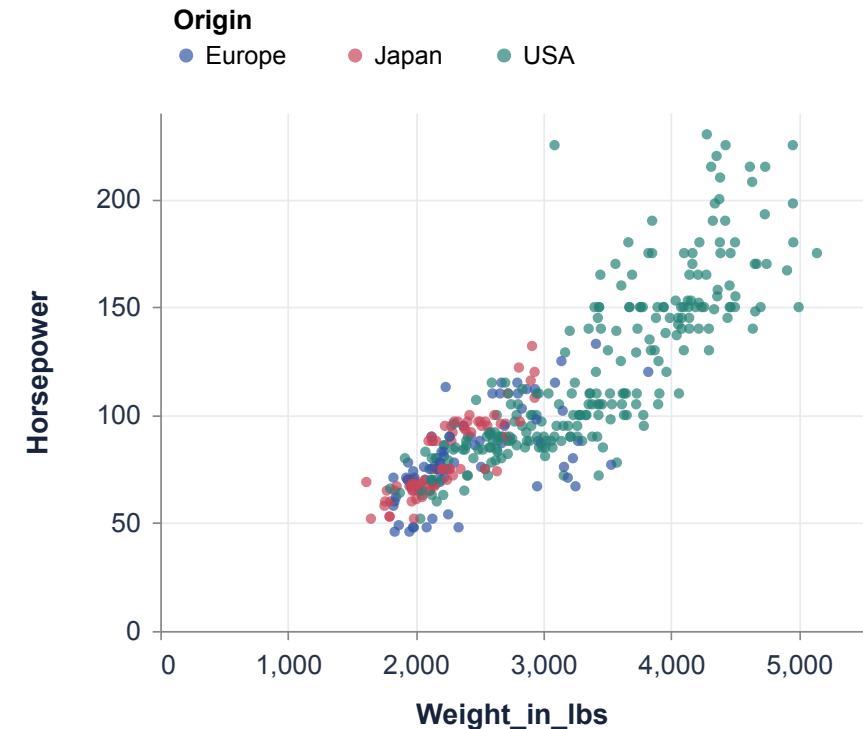
Starting JSON with data
Vega Editor

Which parts of the original specification did you change?

A complete scatterplot solution

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "circle",
  "encoding": {
    "x": {
      "field": "Weight_in_lbs",
      "type": "quantitative"
    },
    "y": {
      "field": "Horsepower",
      "type": "quantitative"
    },
    "color": {
      "field": "Origin",
      "type": "nominal"
    }
  }
}
```



Source: [Vega Datasets · Cars · Full JSON with data](#)

Weight and Horsepower control position. Origin controls color.

Reference: [Vega-Lite · Encoding](#)

Aggregation prepares summary values for a chart

Aggregation combines multiple records into a summary value for each group, using an operation such as mean, sum, or count.

Question: average MPG by Origin

Four rows from the Cars dataset:

Origin	MPG
USA	18
USA	15
Japan	24
Europe	25

Means from the full dataset

Average valid MPG values within each Origin.

Origin	Mean MPG
Europe	27.9
Japan	30.5
USA	20.1

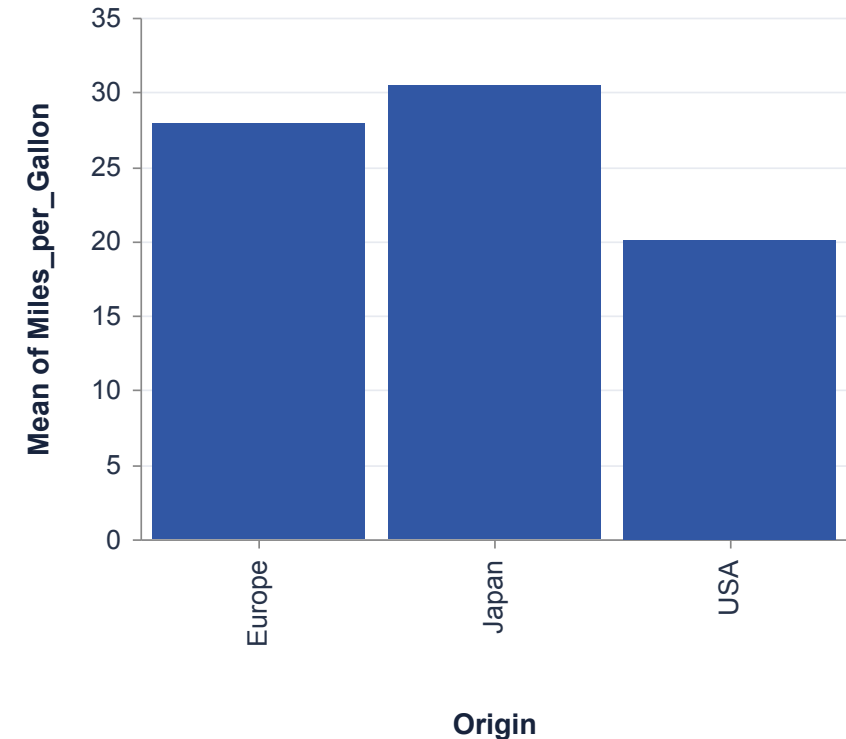
Reference: [Vega-Lite](#) · [Aggregate](#)

Data: [Vega Datasets](#) · [Cars](#)

A bar chart of mean MPG by Origin

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "bar",
  "encoding": {
    "x": {
      "field": "Origin",
      "type": "nominal"
    },
    "y": {
      "aggregate": "mean",
      "field": "Miles_per_Gallon",
      "type": "quantitative"
    }
  }
}
```



Source: [Vega Datasets](#) · Cars · Full JSON with data

Origin is the unaggregated grouping field. The y encoding computes one mean MPG per Origin.

Reference: [Vega-Lite](#) · Aggregate

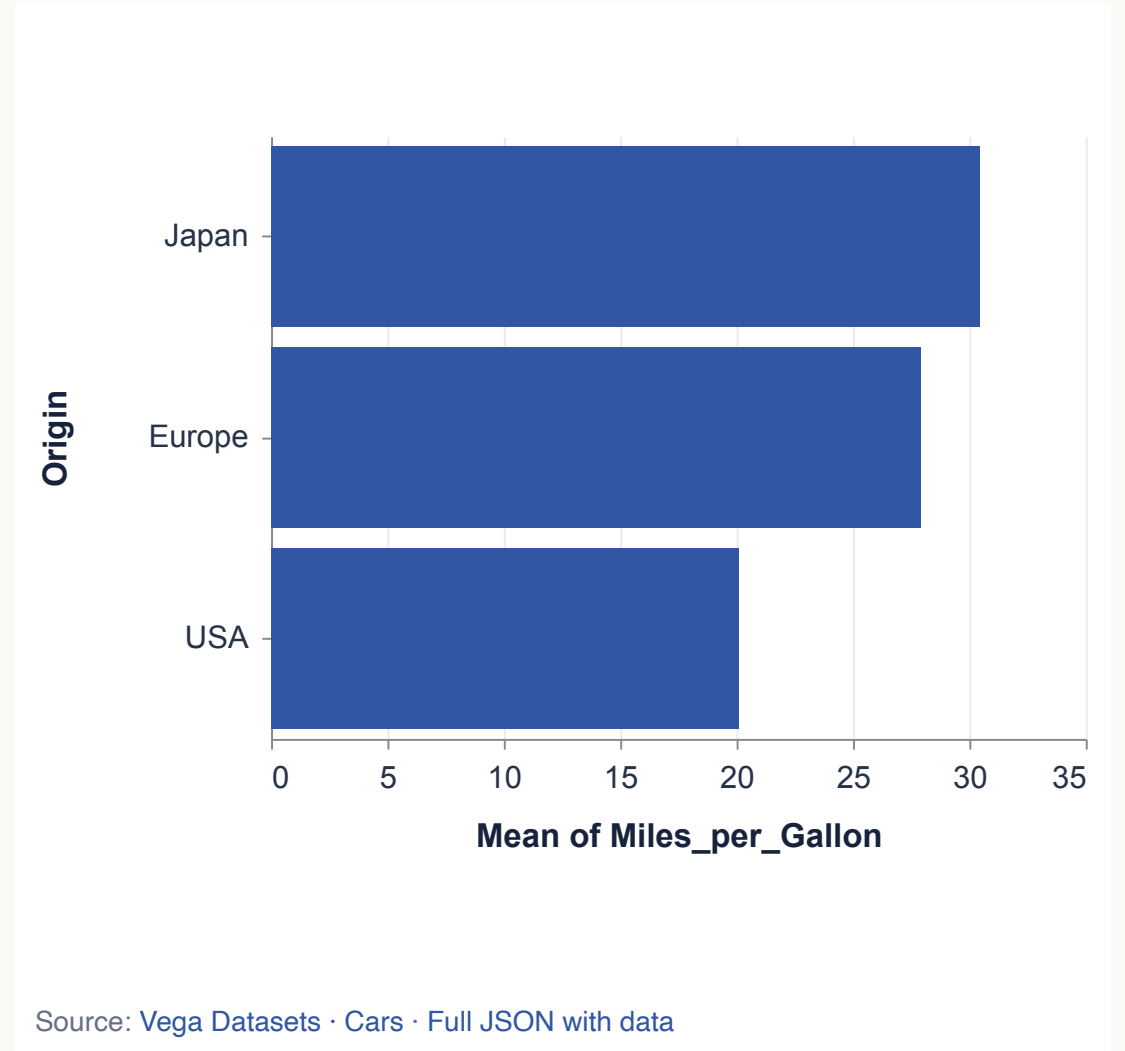
Swapping x and y makes horizontal bars

REPLACE THE ENCODING OBJECT

```
{
  "encoding": {
    "x": {
      "aggregate": "mean",
      "field": "Miles_per_Gallon",
      "type": "quantitative"
    },
    "y": {
      "field": "Origin",
      "type": "nominal",
      "sort": "-x"
    }
  }
}
```

`"sort": "-x"` orders Origin by descending mean MPG.

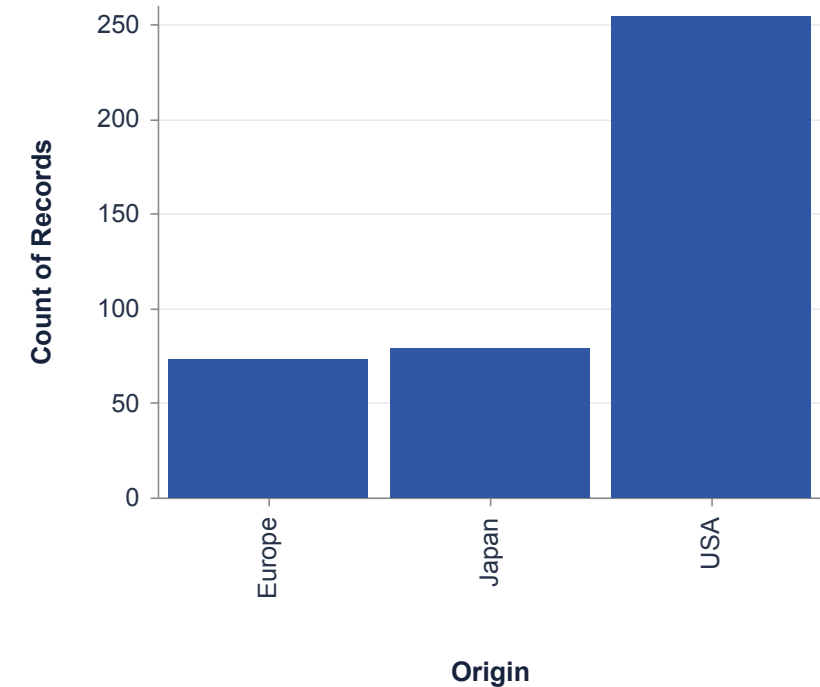
Reference: [Vega-Lite · Sort](#)



Count gives the number of records in each group

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "bar",
  "encoding": {
    "x": {
      "field": "Origin",
      "type": "nominal"
    },
    "y": {
      "aggregate": "count",
      "type": "quantitative"
    }
  }
}
```



Source: [Vega Datasets](#) · Cars · Full JSON with data

Count is the number of records in a group. It does not require a field: Origin defines the groups, and y counts their records.

Reference: [Vega-Lite](#) · Aggregate

Common aggregates compute different summaries

The `aggregate` operation determines which summary value each group produces.

Operation	Result for each group
<code>count</code>	Number of records (no field required).
<code>valid</code>	Number of field values excluding null, undefined, and NaN.
<code>distinct</code>	Number of different field values.
<code>sum</code>	Total of the numeric field values.
<code>mean</code> / <code>average</code>	Arithmetic average. Both names perform the same operation.
<code>median</code>	Middle value after sorting, or the mean of the two middle values.
<code>min</code> / <code>max</code>	Smallest / largest field value.

Reference and full operation list: [Vega-Lite](#) · [Aggregation](#).

Binning groups numeric values into intervals

Binning partitions a numeric range into intervals and assigns each valid value to an interval.

For a Horsepower histogram, these Cars values fall into 20-horsepower intervals:

Horsepower value	Interval	Interval rule
87, 95	80–100	$80 \leq \text{Horsepower} < 100$
130	120–140	$120 \leq \text{Horsepower} < 140$
165	160–180	$160 \leq \text{Horsepower} < 180$

Binning creates the intervals. **Count** then gives the number of cars in each interval.

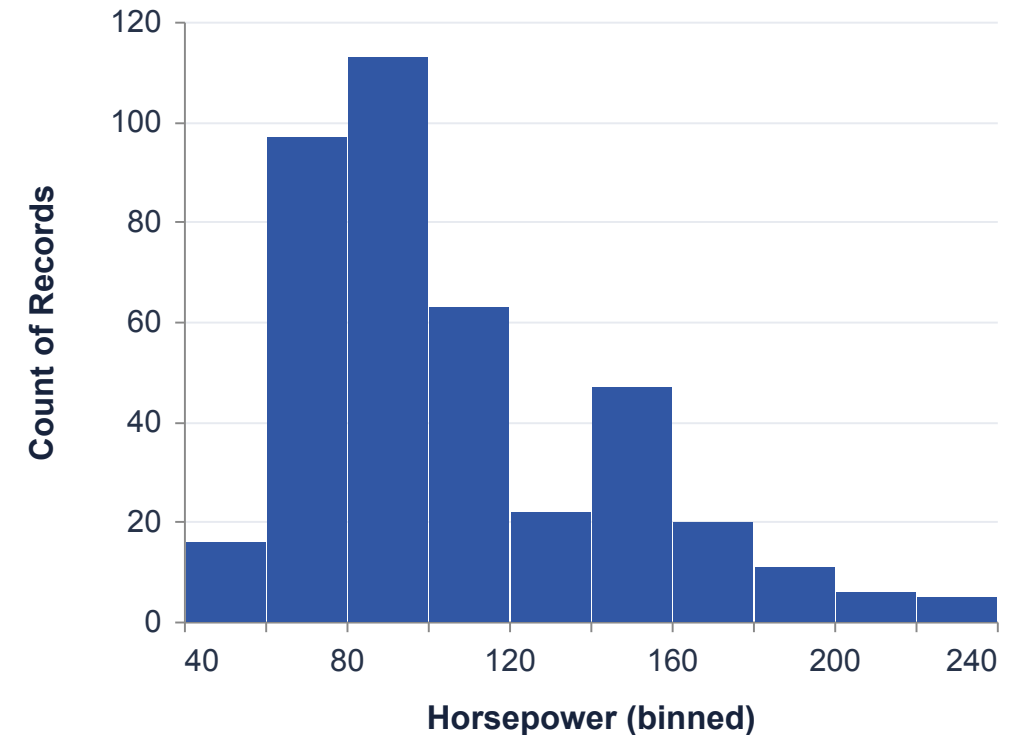
Reference: [Vega-Lite · Bin](#)

Values shown: [Vega Datasets · Cars](#)

Bin and count produce a histogram

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "bar",
  "encoding": {
    "x": {
      "bin": true,
      "field": "Horsepower",
      "type": "quantitative"
    },
    "y": {
      "aggregate": "count",
      "type": "quantitative"
    }
  }
}
```



Source: [Vega Datasets · Cars · Full JSON with data](#)

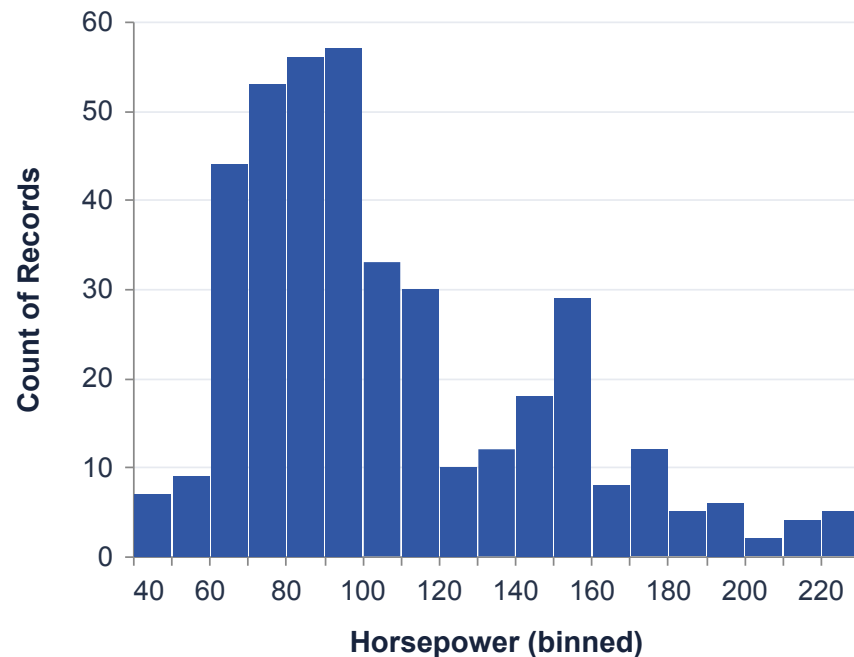
bin:true groups Horsepower into automatically chosen intervals. Count supplies the bar height for each interval.

Reference: [Vega-Lite · Bin](#)

Bin width changes the detail in a histogram

IN X: INTERVALS 10 HORSEPOWER WIDE

```
"bin": {"step": 10}
```

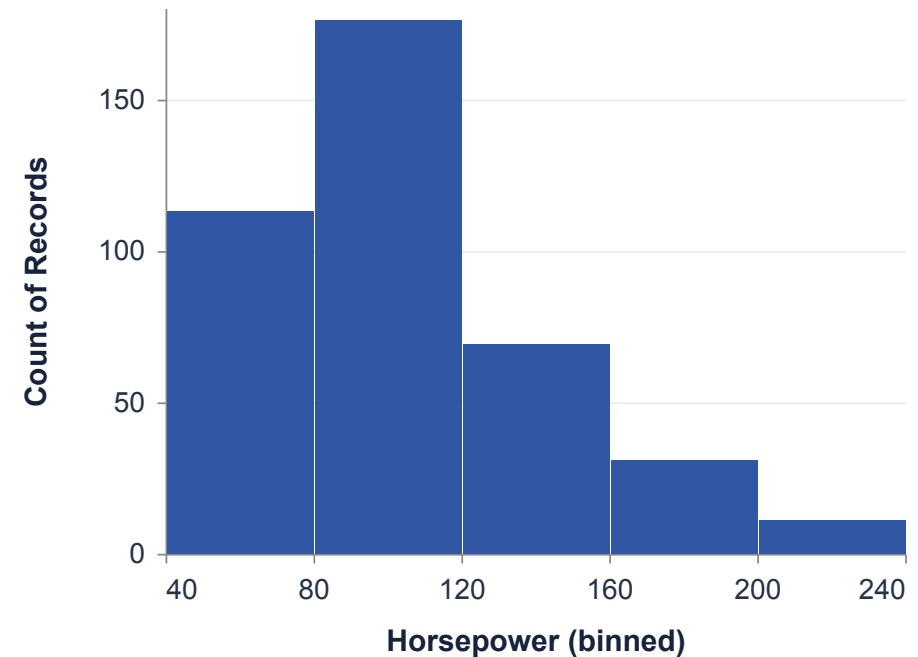


Source: [Vega Datasets](#) · [Cars](#) · [Full JSON with data](#)

Narrower intervals show more local variation.

IN X: INTERVALS 40 HORSEPOWER WIDE

```
"bin": {"step": 40}
```



Source: [Vega Datasets](#) · [Cars](#) · [Full JSON with data](#)

Wider intervals combine more values into each bar.

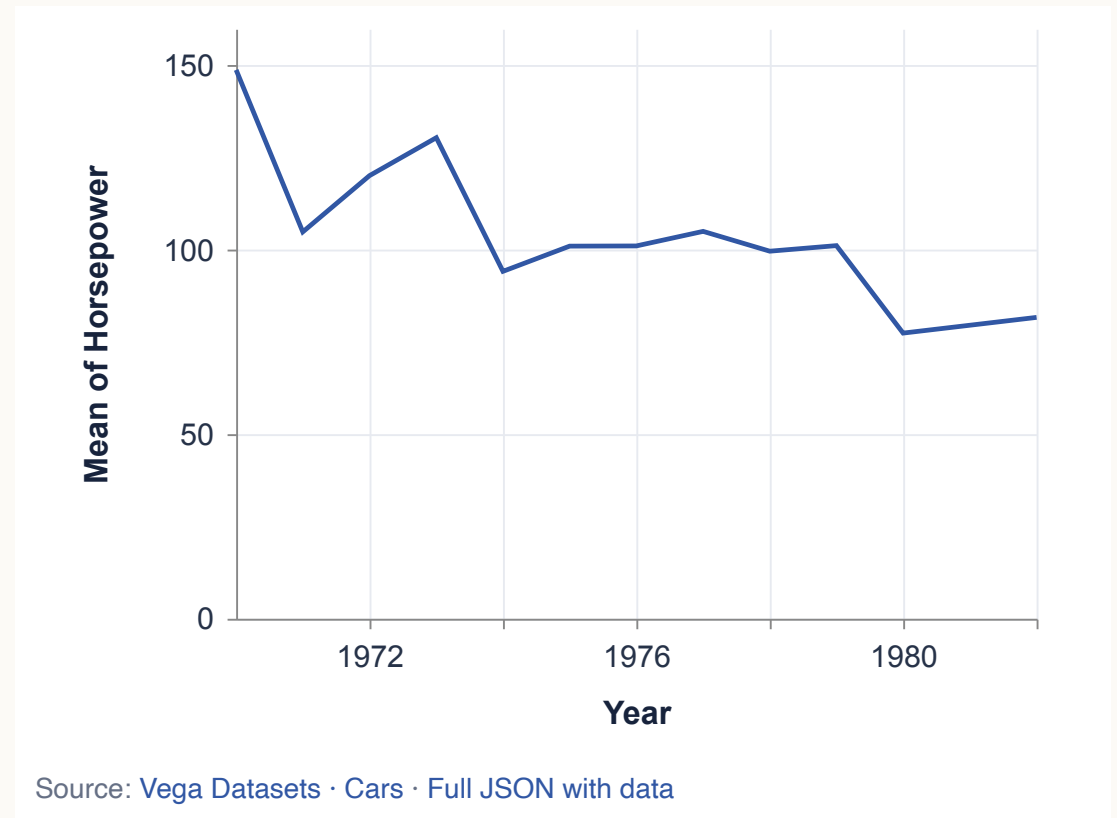
A line connects values along an ordered axis

A line mark connects a sequence of positions with line segments. It shows change along an ordered axis, such as time.

First four yearly means

Year	Mean Horsepower
1970	148.9
1971	104.9
1972	120.2
1973	130.5

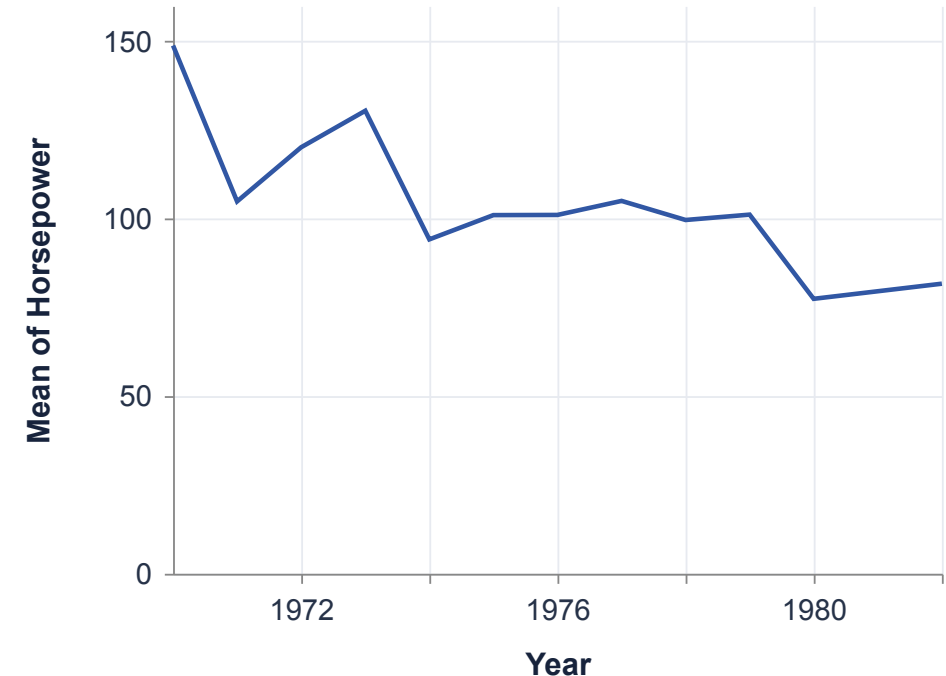
Reference: [Vega-Lite](#) · [Line](#)



A line chart of mean horsepower by year

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "line",
  "encoding": {
    "x": {
      "field": "Year",
      "type": "temporal"
    },
    "y": {
      "aggregate": "mean",
      "field": "Horsepower",
      "type": "quantitative"
    }
  }
}
```



Source: [Vega Datasets · Cars · Full JSON with data](#)

Use the actual field Year as temporal x. Aggregate Horsepower with mean on y, and connect the summaries with line.

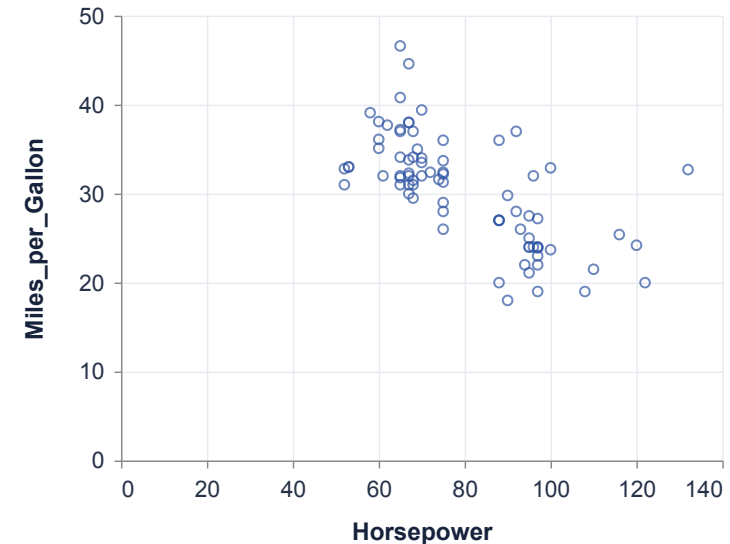
Reference: [Vega-Lite · Line](#)

A filter selects records before plotting

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "transform": [
    {
      "filter": {"field": "Origin",
        "equal": "Japan"}
    }
  ],
  "mark": "point",
  "encoding": {
    "x": {"field": "Horsepower",
      "type": "quantitative"},
    "y": {"field": "Miles_per_Gallon",
      "type": "quantitative"}
  }
}
```

A **filter** keeps records that satisfy a condition and removes the rest from the view.



Source: [Vega Datasets](#) · Cars · Full JSON with data

`transform` is optional and sits at the same level as `data`, `mark`, and `encoding`. Its operations run before encoding.

Reference: [Vega-Lite](#) · Filter

Common transforms prepare data for encoding

The `transform` array runs operations in order. Each operation has its own parameters.

Operation	What it does
<code>filter</code>	Keep records that match a condition.
<code>calculate</code>	Add a field computed from an expression.
<code>aggregate</code>	Replace each group with summary values.
<code>bin</code>	Assign numeric values to intervals.
<code>timeUnit</code>	Derive time units, such as year or month, from dates.
<code>fold</code>	Turn selected columns into key/value rows.
<code>lookup</code>	Add matching fields from another data source.
<code>window</code>	Compute ranks, running totals, or moving averages.

For `aggregate` and `bin`, transform syntax differs from encoding shorthand.

Operation names link to their syntax. Full list: [Vega-Lite · Transformation](#).

Filter conditions select which records to keep

Use these condition objects as the value of `filter` inside `transform`.

Condition	Meaning	Example condition object
<code>equal</code>	Equals	<code>{"field": "Origin", "equal": "Japan"}</code>
<code>not</code>	Negation	<code>{"not": {"field": "Origin", "equal": "Japan"}}</code>
<code>lt</code> / <code>lte</code>	$< / \leq$	<code>{"field": "Cylinders", "lt": 6}</code>
<code>gt</code> / <code>gte</code>	$> / \geq$	<code>{"field": "Cylinders", "gte": 6}</code>
<code>range</code>	Inclusive interval	<code>{"field": "Horsepower", "range": [80, 120]}</code>
<code>oneOf</code>	One of a set	<code>{"field": "Origin", "oneOf": ["Japan", "Europe"]}</code>
<code>valid</code>	Valid value	<code>{"field": "Horsepower", "valid": true}</code>

For “not equal”, wrap `equal` in `not`. There is no `unequal` predicate. References: [Filter](#) · [Predicates](#).

Practice: draw a new histogram

Show the distribution of **engine displacement**.

- Field: `Displacement`
- Interval width: **50 cubic inches**
- Height: number of car records in each interval

Write and run the complete specification.

Check your result

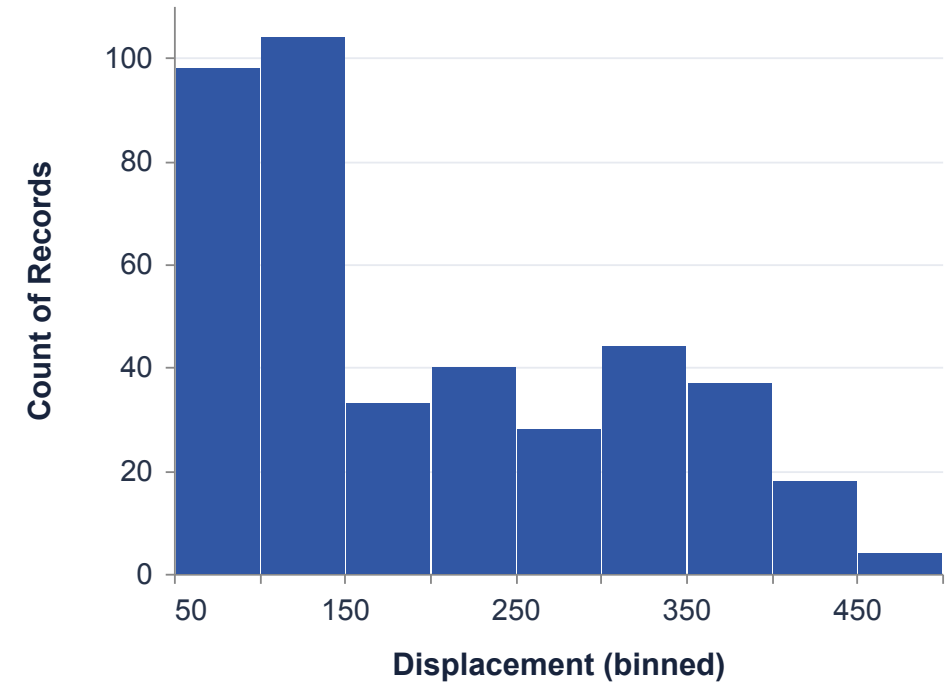
1. Which mark produces the bars?
2. Where do `field`, `type`, and `bin` belong?
3. What should the y encoding contain?
4. Are intervals 50 units wide?

[Starting histogram JSON with data Vega Editor](#)

A complete histogram solution

COMPLETE SPECIFICATION

```
{
  "data": {"url": "data/cars.json"},
  "mark": "bar",
  "encoding": {
    "x": {
      "bin": {"step": 50},
      "field": "Displacement",
      "type": "quantitative"
    },
    "y": {
      "aggregate": "count",
      "type": "quantitative"
    }
  }
}
```



Source: [Vega Datasets](#) · Cars · Full JSON with data

Displacement is quantitative. step:50 creates 50-unit intervals, and count gives the number of car records in each interval.

Reference: [Vega-Lite](#) · Bin

Four chart patterns to reuse

Chart	mark	x	y
Scatterplot	point	A quantitative field	Another quantitative field
Group bar chart	bar	A nominal field	A quantitative field with aggregate: "mean"
Histogram	bar	A quantitative field with bin: true	aggregate: "count" and quantitative type
Line chart	line	A temporal field	A quantitative field with a mean when needed

A color encoding can add a group field. With aggregation, an additional unaggregated field also changes the grouping.

Reference: [Vega-Lite · Encoding](#)

Build, run, and revise a specification

1. Load the data.

Inspect field names and example values.

2. Choose a mark; define its channels.

Use field/type or a fixed value in encoding.

3. Add operations when needed.

Aggregate, bin, or filter for the chart.

4. Run and inspect the result.

Check axes, groups, and summaries.

If the chart looks wrong

Check the JSON structure, exact field names, field types, and the meaning of each aggregate or bin.

Keep these references

[Vega Editor](#)

[Vega-Lite documentation](#)

[Example gallery](#)

Next lecture: combine views and add interaction.